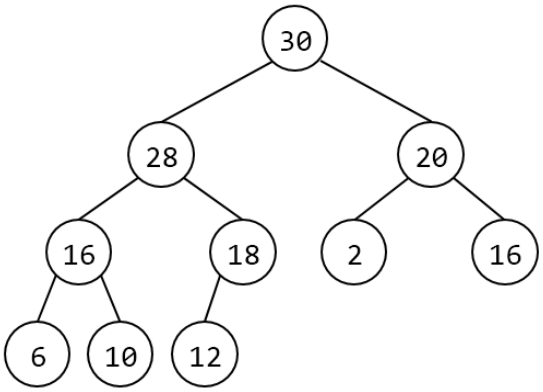
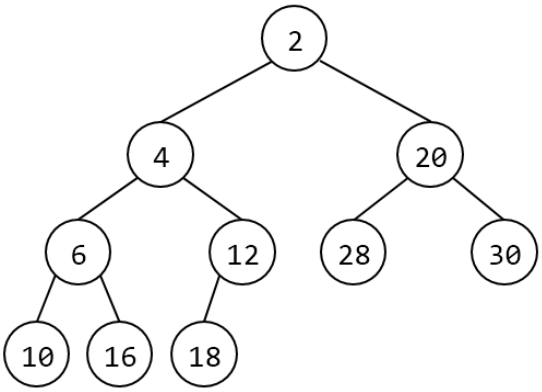
树形数据结构

二叉堆

堆是一棵完全二叉树，如果每一个结点的值大于或等于左右子结点的值，则称之为最大堆（大顶堆或大根堆）；若每一个结点的值小于或等于左右子结点的值，则称之为最小堆（小顶堆或小根堆）。



大根堆



小根堆

而二叉堆（以大根堆为例）有两个基本的操作：

出队。

出队时，堆顶（队头）出队，最后一个元素代替栈顶的位置，重新调整为堆。我们来看下其调整的过程。

堆顶 30 出队， 最后一个元素 12 代替堆顶。出队后，除了堆顶，其他结点都满足最大堆的定义，只需堆顶执行**下沉**操作，即可调整为堆。**下沉** 指堆顶与左右子结点比较，若比子结点大，则已调整为堆了；若比子结点小，则与较大的子结点交换，交换到新的位置后继续向下比较，从根结点一直比较到叶子结点。

堆顶 **下沉** 过程：

1. 堆顶 12 与两个子结点 28， 20 比较，比子结点小，与较大的子结点 28 交换。
2. 12 再和两个子结点 16，18 比较，比子结点小，也较大的子结点 18交换。
3. 比较到叶子结点时停止，已调整为堆结构

代码参考：

```
void Sink(int k,int n) {     //下沉操作
    while(2*k <=n) {         //当前不是叶子结点
        int j=2*k;           //左结点的编号
        if(j<n && r[j]<r[j+1]) {     //如果有右结点的话且右结点的值>左结点
            j++;                     //确定交换的结点 j
        }
        if(r[k]>=r[j]){      //如果父结点大于子结点，则已调整为堆了
            break;
        }else {              //否则父结点与最大的子结点j 进行交换
            swap(r[k],r[j]);
        }
        k=j;                 //结点k 覆盖为 j,继续下沉操作
    }
}
```

```
void pop(int n) {
    cout<<r[1]<<endl;    //输出堆顶
    r[1]=r[n--];         //最后一个元素替代堆顶，同时总结点-1
    Sink(1,n);           //进行下沉操作
}
```

入队

入队时，将新元素放入最后一个元素之后，重新调整为堆。

入队后除了新入队的元素，其他结点都满足最大堆的定义，只需新元素执行 **上浮** 操作，即可调整为堆。 **上浮**指新元素与父结点比较，若小于或等于父结点，则调整为堆； 若比父结点大，则与父结点交换，交换到新位置后，继续向上比较，从叶子一直比较到根。

新元素 **上浮** 的过程：

1. 新元素 29 和其父结点 18 比较，比父结点大，与父结点交换。
2. 29 再和其父结点 28 比较，比父结点大，与父结点交换。
3. 29 再和其父结点（根）30比较，比父结点小，已调整为堆结构。

代码参考：

```
void Swim(int k,int n) {              //上浮操作
    while(k>1 && r[k]>r[k/2]) {       //当结点k>1，表示其有父结点，如果子结点k 大于父结点k/2，则交换。
        swap(r[k],r[k/2]);
        k=k/2;                        //结点上浮，即判断父结点k/2
    }
}
void push(int n,int x) {
    r[++n]=x;         //在最后添加新元素
    Swim(n);          //执行上浮操作
}
```

P3378 【模板】堆

题目简述

给定一个数列，初始为空，支持下面三种操作：

1. 给定一个整数 x，请将 x加入到数列中。
2. 输出数列中最小的数。
3. 删除数列中最小的数（如果有多个数最小，只删除 1 个）。

对这个数列进行 n 次操作。

数据范围

$1 \leq n \leq 10^6$，$1 \leq x < 2^{31}$

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
const int N=1000010;
int a[N];   //序列大小
int p;
int n;

void Sink(int k) {  //下沉操作，从根节点开始
    while(k*2 <=n) {
        int j= k*2; //左结点
        if(j<n && a[j]>a[j+1])  j++;    //如果有右结点且右结点较大时，较大值编号为j+1
        if(a[k]<=a[j])  break;
        else    swap(a[k],a[j]);
        k=j;
    }
}

void Swim(int k) {  //上浮操作，从叶子结点开始
    while(k>1 && a[k]<a[k/2]) {
        swap(a[k],a[k/2]);
        k=k/2;
    }
}
void push(int x) {
    a[++n]=x;
    Swim(n);
}
void pop() {
    a[1]=a[n--];
    Sink(1);
}
int main() {
    cin>>p;
    while(p--) {
        int op,x;   scanf("%d",&op);
        if(op==1){
            scanf("%d",&x);
            push(x);
        }
        else if(op==2) {
            cout<<a[1]<<endl;
        }
        else if(op==3) {
            pop();
        }
    }
    return 0;
}
```

算法总结

优先队列是利用堆实现的一种特殊队列，堆是按照完全二叉树顺序存储的，有n 个结点的完全二叉树的高度为$log_2 n+1$。出队时，堆顶元素出队，最后一个元素代替堆顶，新的堆顶从根下沉到叶子，最多达到树的高度，时间复杂度是$O(logn)$；入队时，新元素从叶子上浮到根，最多达到树的高度，时间复杂度也为$O(logn)$。

优先队列

普通的队列是一种先进先出的数据结构，从队尾入队，从队头出队。在优先队列中，元素被赋予优先级，优先级高的元素先出队。优先队列的实现原理是通过堆实现的，在实际的算法实现中，可以直接调用$C++$的STL 函数`priority_queue`。

优先队列`priority_queue` 的成员函数如下：

`empty()`	`pop()`	`push()`	`top()`	`size()`
判断队列是否为空	出队	入队	取栈顶元素	队列中元素的个数

优先队列的用法：

`priority_queue<int, vector<int>, cmp>`

其中，第 1 个参数为数据类型， 第2 个参数为容器类型，第 3 个参数为比较函数，后两个参数根据需要也可以省略。

`priority_queue<int>` `//参数为整型，默认优先级最大值优先`

如何控制优先队列的优先级？若不是最大值优先，则可以采用下面的方法：

```
#include<iostream>
#include<functional>
#include<queue>
#include<vector>
using namespace std;
//自定义优先级1 ，数值类型
struct cmp {
    bool operator()(int &a,int &b) {
        return a<b; //最大值优先
        // return a>b;  最小值优先
    }
};
//自定义优先级2， 结构体类型
struct node {
    int x,y;
    node() {};  //重载函数
    node(int _x,int _y) {
        x=_x;   y=_y;
    }
};
// 重载 < 小于号
bool operator <(const node &a, const node &b)  {
    return a.x>b.x; //按成员x最小值优先
}
int a[]={15,7,32,26,97,48,36,89,6,49,67,0};
int b[]={1,2,5,6,9,8,6,9,7,19,27,0};
int main() {
    cout<<"自定义优先级1,数值型:"<<endl;
```

```
    priority_queue<int,vector<int>,cmp> que1;    //采用默认优先级构造队列
    for(int i=0;a[i];i++)   que1.push(a[i]);
    while(!que1.empty()) {
        cout<<que1.top()<<" ";
        que1.pop();
    }
    cout<<endl;

    cout<<"自定义优先级2,结构体型:"<<endl;
    priority_queue<node> que2;
    for(int i=0;b[i];i++)   que2.push(node(a[i],b[i]));
    while(!que2.empty()) {
        cout<<que2.top().x<<" ";
        que2.pop();
    }
    cout<<endl;
    return 0;
}
```

P3956 [NOIP2017 普及组] 棋盘

题目描述

有一个$m\times m$的棋盘，棋盘上每一个格子可能是红色、黄色或没有任何颜色的。你现在要从棋盘的最左上角走到棋盘的最右下角。

任何一个时刻，你所站在的位置必须是有颜色的（不能是无色的），你只能向上、下、左、右四个方向前进。当你从一个格子走向另一个格子时，如果两个格子的颜色相同，那你不需要花费金币；如果不同，则你需要花费 1个金币。

另外，你可以花费 2个金币施展魔法让下一个无色格子暂时变为你指定的颜色。但这个魔法不能连续使用，而且这个魔法的持续时间很短，也就是说，如果你使用了这个魔法，走到了这个暂时有颜色的格子上，你就不能继续使用魔法；只有当你离开这个位置，走到一个本来就有颜色的格子上的时候，你才能继续使用这个魔法，而当你离开了这个位置（施展魔法使得变为有颜色的格子）时，这个格子恢复为无色。

现在你要从棋盘的最左上角，走到棋盘的最右下角，求花费的最少金币是多少？

输入

第一行包含两个正整数m, n，以一个空格分开，分别代表棋盘的大小，棋盘上有颜色的格子的数量。

接下来的n 行，每行三个正整数x, y, c，分别表示坐标为(x, y)的格子有颜色c 。

其中$c = 1$ 代表黄色，$c = 0$ 代表红色。相邻两个数之间用一个空格隔开。棋盘左上角的坐标为$(1, 1)$，右下角的坐标为(m, m)。

棋盘上其余的格子都是无色。保证棋盘的左上角，也就是$(1, 1)$ 一定是有颜色的。

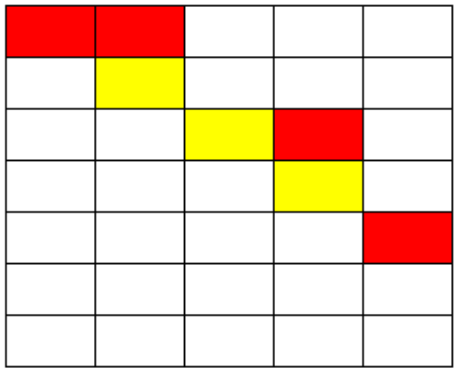
输出

一个整数，表示花费的金币的最小值，如果无法到达，输出-1。

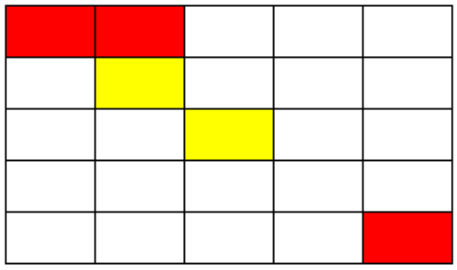
输入输出样例

输入#1	输出#1	输入#2	输出#2
5 7 1 1 0 1 2 0 2 2 1 3 3 1 3 4 0 4 4 1 5 5 0	8	5 5 1 1 0 1 2 0 2 2 1 3 3 1 5 5 0	-1

算法分析：



样例1



样例2

样例1解释：

从$(1,1)$开始，走到$(1,2)$不花费金币

从$(1,2)$向下走到$(2,2)$花费 **1** 枚金币

从$(2,2)$施展魔法，将$(2,3)$变为黄色，花费 **2** 枚金币

从$(2,2)$走到$(2,3)$不花费金币

从$(2,3)$走到$(3,3)$不花费金币

从$(3,3)$走到$(3,4)$花费 **1** 枚金币

从$(3,4)$走到$(4,4)$花费 **1** 枚金币

从$(4,4)$施展魔法，将$(4,5)$变为黄色，花费**2** 枚金币

从$(4,4)$走到$(4,5)$不花费金币

从$(4,5)$走到$(5,5)$花费 **1** 枚金币

共花费 **8** 枚金币。

从题目上，如果采用最直接的方式，就是盲目的广度搜索(BFS)，压入队列的元素需要哪些信息呢？从题目中可以得出，压入队列需要的信息有：坐标信息x,y，走到当前需要的金币$coin$，当前的颜色$color$，是否施展过魔法$flag$。队列的元素需要那么多元素，采用结构体的形式将他们绑定在一个大元素内。

接着，继续分析：如果盲目的压入队列，我们需要**反复搜索到某个点**，反复比较求出当前格子的最优值。很明显时间复杂度太高了。我们需要想一种 **“聪明”** 的搜索方式。

题目求最值，我们每次扩展的点应该是最优值（贪心）策略，比如$(2,2)$这个点最优值应该是由$(1,2)$扩展来的，而不是由$(2,1)$扩展来的。但是后面的路径有可能会经过$(1,2)$，此时$(1,2)$，$(2,1)$都花费了 **1** 个金币，我们都把他们压入队列，每次**取最少金币**的元素进行外围扩展，直到扩展到终点，那么就一定是最优值了。

分析到这里，队列的头部每次都要保持最优值，怎么办？ 这里我们可以采用优先队列来解决。重载比较函数，让队首保持金币最少。

最后就是细节分析：当前格子是否施展魔法的细节判断了。

如果当前格子施展了魔法（即要么是红色，要么是黄色）

- 如果下一个格子是无色的，则该格子不能走，跳过。（因为不能再施展魔法走了）
- 如果下一个格子是同色，则不需要花费。
- 如果下一个格子是不同色，则需要累加 1 个金币

如果当前格子没有施展魔法（当前格子也是要么是红色，要么是黄色）

- 如果下一个是无色的，则需要将其变成同色，累加 2 个金币，并**标记该格子**施展过魔法 $flag = true$
- 如果下一个格子是同色的，则不需要花费
- 如果下一个格子是不同色，则累加 1 个金币

代码参考：

```cpp
#include <iostream>
#include <queue>
#include <cstdio>
#include <algorithm>
#include <cstring>
using namespace std;
const int N=110;
const int dx[5]={0, 0,0,-1,1};
const int dy[5]={0,-1,1, 0,0};

struct node {
    int x,y;
    int coin;
    bool flag;  //默认没有使用魔法
}a[N];
bool operator <(const node &a, const node &b) {
    return a.coin > b.coin;     //金币小的优先级小。
}
int g[N][N];
bool vis[N][N]; //标记是否走过
priority_queue<node> que;

int main() {
    int m,n;    cin>>m>>n;
    memset(g,-1,sizeof g);  //初始化 -1
    for(int i=1;i<=n;i++){
        int x,y,z;
        cin>>x>>y>>z;
        g[x][y]=z;

    }
    node tmp;
    tmp.x = 1; tmp.y = 1;   tmp.coin=0; tmp.flag=false;     //第一个元素
```

```
    que.push(tmp);
    vis[1][1]=true;
    while(!que.empty()) {
        node tmp= que.top();    //取堆顶
        que.pop();
        int x=tmp.x, y=tmp.y, z=tmp.coin;
        bool flag = tmp.flag;   //队头元素是否施展过魔法

        //cout<<x<<" "<<y<<" "<<z<<" "<<flag<<endl;
        if(x==m && y==m){   //找到答案了
            cout<<z;
            return 0;
        }

        for(int i=1;i<=4;i++){  //4个方向搜索
            node tmp1;  //新建一个结构体并初始化数据
            tmp1.x =x+ dx[i];   tmp1.y =y+dy[i];    tmp1.coin =z;
tmp1.flag=false;
            int nx=tmp1.x;  int ny=tmp1.y;
            if(nx<1 || nx>m || ny<1 || ny>m || vis[nx][ny]) continue;
            if(flag && g[nx][ny]==-1)   continue;   //父结点施展过魔法，要么是红
色要么是黄色
            if(flag) {
                if(g[x][y]== g[nx][ny]) tmp1.coin =z;
                else tmp1.coin =z+1;    //不同颜色需要+1
            }
            if(!flag) { //队头没有使用过魔法
                if(g[nx][ny]==-1) {     //空格
                    g[nx][ny] = g[x][y];    //将当前格子变色
                    tmp1.coin = z+2;    //金币需要+2
                    tmp1.flag = true;
                }
                else if(g[nx][ny]!=g[x][y]){
                    tmp1.coin = z+1;    //不同颜色需要花费金币+1
                }
            }
            vis[nx][ny]=1;  //标记扩展的子结点已经走过
            que.push(tmp1);
        }
    }
    cout<<-1;
    return 0;
}
```

例题推荐

P3887 [GDOI2014]世界杯

P1090 [NOIP2004 提高组] 合并果子

P1334 瑞瑞的木板

P1628 合并序列

P1878 舞蹈课

P1717 钓鱼

P1168 中位数

P1801 黑匣子

P2168 [NOI2015] 荷马史诗

线段树

有这样一类RMQ（**R**ange **M**inimum / **M**aximum Query）问题，求区间最大值或最小值。设有长度为n的数列$\{a_1, a_2, \ldots, a_n\}$，需要进行以下操作。

- 求最值：给定$i, j \leq n$,求$\{a_1, a_2, \ldots, a_n\}$区间内的最值
- 修改元素：给定k和x，把a_k修改成 x.

如果用普通数组存储数列，上面两个操作中，求最值的复杂度是$O(n)$，修改是$O(1)$。如果有m次“修改元素+查询最值”，那么总复杂度是$O(mn)$。如果m和n比较大，例如100000以上，那么整个程序的复杂度是10^{10}的数量级。这个复杂度在竞赛中是不可接受的。

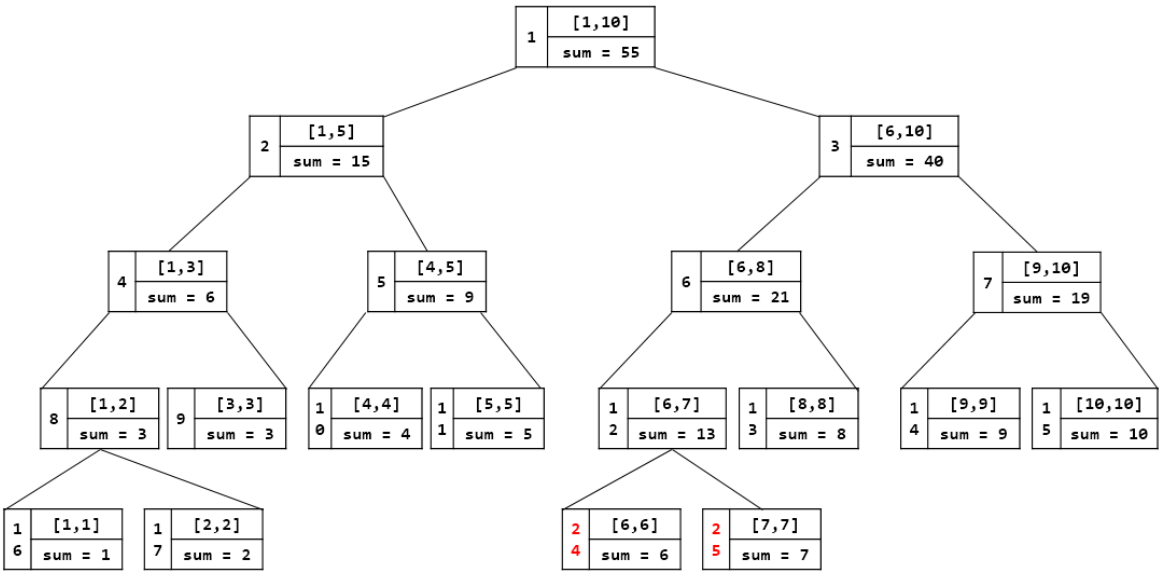
除了RMQ问题以外，类似的还有求区间和问题。对于数列$\{a_1, a_2, \ldots, a_n\}$，先更改某些数的值，然后给定$i, j \leq n$，求$sum = a_i + a_{i+1} + \ldots + a_j$的区间和。对于单个更改或者求和，很容易写出$O(n)$的算法；如果更改和询问的操作总次数是$m$，那么整个程序的复杂度是$O(mn)$。和$RMQ$一样，这样的复杂度也是不行的。

概念

线段树是一个查询和修改复杂度都为$log(n)$的数据结构。主要用于数组的单点修改和区间求和。线段树是建立在线段（或区间）基础上的**二叉树**，树的每个结点代表一条线段$[L, R]$。

设 $a[11] = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$， 我们来构建它的线段树，如图所示：

构建



线段树是二叉树，一个区间每次被折一半往下分，所以最多分$\log_2 n$次就到达最底层。当需要查找一个点或者区间的时候，顺着结点往下找，最多$\log_2 n$次就能找到。这就是线段树效率高的原因，使用了**二叉树折半查找**的方法。

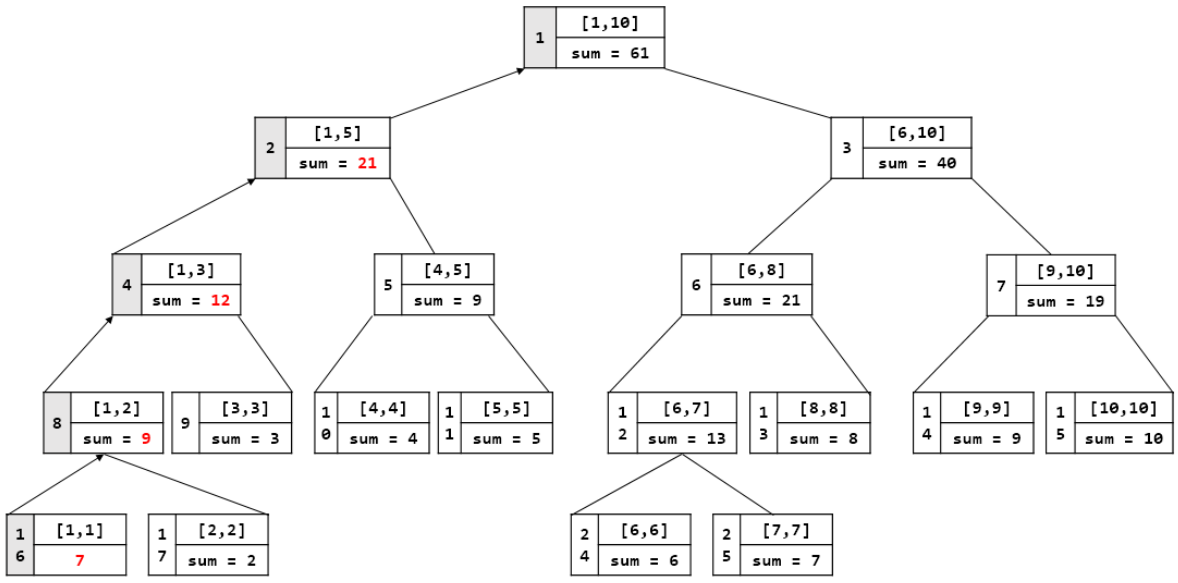
注意：从二叉树视角可以看出：N个叶节点的满二叉树有
$N + N/2 + N/4 + \ldots + 2 + 1 = 2N - 1$个节点。因为在上述存储方式下，最后还有一层产生了空余，所以保存线段树的数组长度要不小于$4N$才能保证不越界

代码参考

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N=100010;
LL a[N],ans;
struct node {
    int l,r;
    LL s,add,mul;
}t[4*N];
void build(int p,int l,int r) {
    t[p].l = l, t[p].r = r;
    if(l==r){       //递归出口
        t[p].s = a[l];
        return ;
    }
    int mid = (l+r)/2;
    build(p*2,l,mid);   //递归左边
    build(p*2+1,mid+1,r);   //递归右边
    t[p].s = t[p*2].s + t[p*2+1].s;   //父节点的和 = 左孩子 + 右孩子
}
//调用：build(1,1,n)   //从第一个节点开始往下构建
```

单点修改

假设要把$a[x]$的值修改为v。比如将$a[1]$修改为7， 我们来看修改的过程：



代码参考

```cpp
void change1(int p,int x,int v) {   //把a[x]的值修改为 v
    if(t[p].l == t[p].r) {          //找到目标结点了，修改其数值
        t[p].s=v;   return ;
    }
    int mid = (t[p].l+t[p].r)/2;
    if(x<=mid)  change(p*2+1,x,v);  //x属于左半区间
    else    change(p*2,x,v);        //x属于右半区间
    t[p].s = t[p*2].s+t[p*2+1].s;   //从下往上更新数据
}
// 调用：  change1(1,1,7)    从根结点1开始往下搜索
```

查询

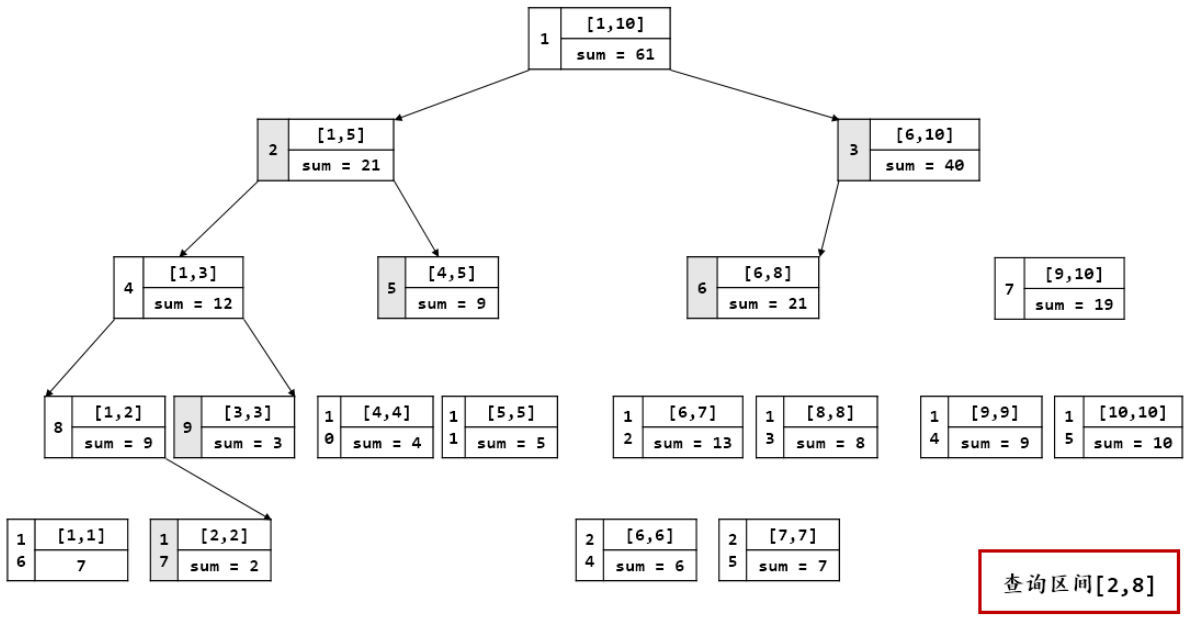
假设我们查询区间$[2,8]$的和，我们发现，线段树上没有这一段区间，但是它可以由其他结点相加而来。那么如何往下搜索相加相应的区间呢？这里需要分 2种大情况：

令：l表示父结点的左区间， r 表示父结点的右区间， mid表示父结点的中点

1. 左右孩子结点都有重叠的可能:
    - 父结点区间$[1,10]$， $mid=5$。 $mid>=2$，左边有重叠需要搜索左边，$8>mid$，右边有重叠需要搜索右边。
    - 父结点区间$[1,5]$， $mid=3$。 $mid>=2$，左边有重叠需要搜索左边，$8>mid$，右边有重叠需要搜索右边。
    - 父结点区间$[1,3]$， $mid=2$。 $mid>=2$，左边有重叠需要搜索左边，$8>mid$，右边有重叠需要搜索右边。
    - 父结点区间$[4,5]$， $mid=4$。 $mid>=2$，左边有重叠需要搜索左边，$8>mid$，右边有重叠需要搜索右边。
2. 只有右孩子结点有重叠的可能:
    - 父结点区间$[1,2]$，$mid=1$。 $mid>=2$，不成立。 $8>mid$，右边有重叠需要搜索右边。
3. 只有左孩子结点有重叠的可能:
    - 父结点区间$[6,10]$， $mid=8$。 $mid>=2$，左边有重叠需要搜索左边，$8>mid$不成立。

递归出口:

如果父结点区间的左右结点都相同，表示已经搜索到底部了，返回相应的值。但是这样虽然简单，但还是走了n步的累加，并没有起到优化效果，我们发现，其实搜到区间$[4,5]$， $[6,8]$就可以返回值，因为这两个区间是包含在区间$[2,8]$里面的，所以我们可以推出条件，如果$l>=2$ 且 $r<=8$，那么就可以直接返回值了。



参考代码

```
LL ask(int p,int l,int r) {
    //递归出口：完全包含，则累加区间值
    if(l<=t[p].l && r>=t[p].r)  return ans+=t[p].s;
    //add_flag(p);
    int mid=(t[p].l+t[p].r)/2;
    if(l<=mid)  ask(p*2,  l,r);      //左结点有重叠
    if(r> mid)  ask(p*2+1,l,r);      //右结点有重叠
    return ans;
}
```

P3374 【模板】树状数组 1

算法分析：

本题是单点修改 + 区间查询。

使用线段树的代码来实现。

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N=500010;
struct node{
    int l,r;
    int val;
}t[4*N];
int a[N];
int n,p;
LL ans;
void build(int p, int l,int r) {    //构建二叉树
    t[p].l=l, t[p].r=r;
    if(t[p].l == t[p].r) {
        t[p].val=a[l];
        return ;
    }
    int mid=(l+r)/2;
    if(l<=mid)  build(p*2, l,mid);
    if(r>mid)   build(p*2+1, mid+1,r);
    t[p].val = t[p*2].val + t[p*2+1].val;   //当前结点的值 = 左右孩子结点累加的值
}

void change(int p,int x,int k) {    //单点修改
    int l=t[p].l, r=t[p].r;
    if(l == r) {
        t[p].val+= k;
        return ;
    }
    int mid=(l+r)/2;
    if(x<=mid)  change(p*2,x,k);    //如果x在左边，则只搜索左半边，否则搜索右半边
    else change(p*2+1, x,k);
    t[p].val = t[p*2].val + t[p*2+1].val;
}

void query(int p,int l,int r) {
    if(t[p].l>=l && t[p].r<=r) {    //累加符合的区间
```

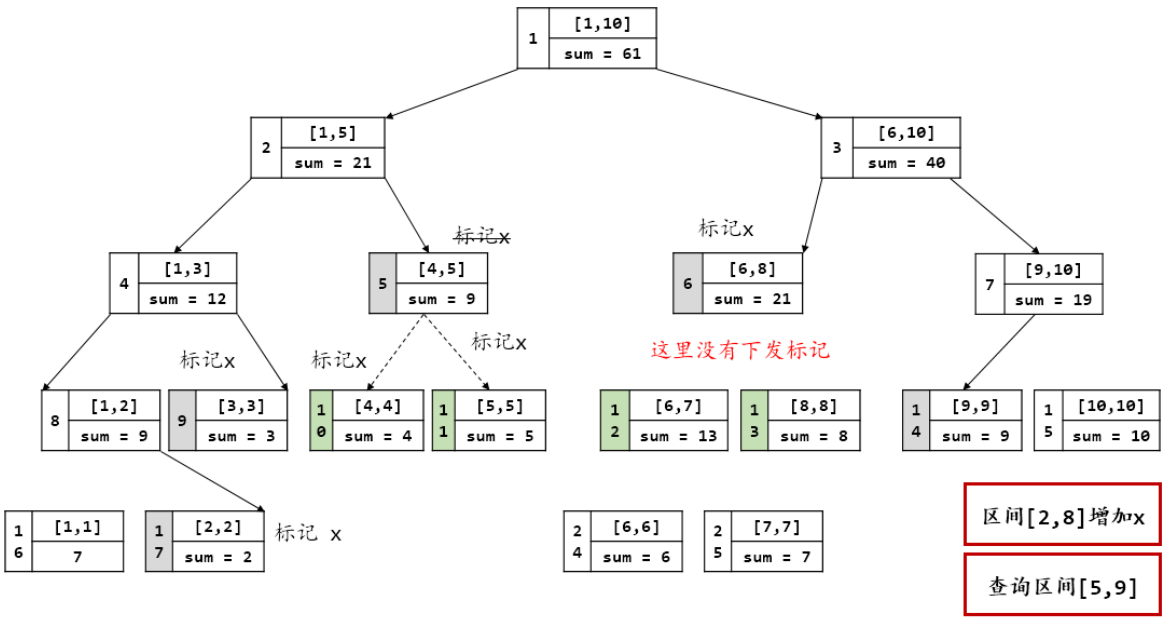
```
        ans = ans+t[p].val;
        return ;
    }
    int mid=(t[p].l + t[p].r)/2;
    if(mid>=l)  query(p*2,l,r);      //左边有重叠的
    if(r>mid)   query(p*2+1,l,r);    //右边有重叠的

}
int main() {
    scanf("%d%d",&n,&p);
    for(int i=1;i<=n;i++)   scanf("%d",&a[i]);
    //构建线段树（二叉树）
    build(1,1,n);

    while(p--) {
        int op,x,y;
        scanf("%d%d%d",&op,&x,&y);
        if(op==1) {
            change(1,x,y);
        }
        else if(op==2) {
            ans=0;  query(1,x,y);
            cout<<ans<<endl;
        }
    }
    return 0;
}
```

区间修改 + 懒惰标记

如果单点修改增加到区间修改，采用朴素的办法，将区间展开成单点修改，那么其时间复杂度是$O(nlogn)$，如果有m次的区间修改，其总的时间复杂度为$O(mnlogn)$。采用与差分算法相似的思路，在需要修改的地方做以下标记，多次标记就累加即可。在遇到区间和询问时再统一递归计算相关区间即可。时间复杂度可以优化到$O(m*logn*logn)$



我们来模拟一下这个过程，修改区间$[2,8]$，那么根据查询区间的方式，我们把结点$5,9,17,6$都做了标记x。

查询区间$[5,9]$时：

1. 当查询到区间$[4,5]$时，发现该点有 **标记**， 把父结点的 **标记** 下发下去，同时**更新子结点的数值** 和 对子结点做 **标记。**最后**清零**父结点的 **标记**
2. 当查询到区间$[6,8]$时，发现该点有 **标记**，但是该区间是包含在区间$[5,9]$的，那么不需要下发了，直接**更新父结点的数值**即可，父结点的 **标记** 继续保留
3. 当查询到区间$[9,10]$时，发现该点没有 **标记**，则按正常搜索下来即可。

标记 代码

```
//区间修改--懒惰标记值，
void spread(int p) {
    if(tree[p].flag) {  //如果该节点有标记，则进行该区间的更新
        tree[p*2].value += tree[p].flag *(tree[p*2].right - tree[p*2].left +1);
        tree[p*2+1].value += tree[p].flag *(tree[p*2+1].right - tree[p*2+1].left +1);

        tree[p*2].flag += tree[p].flag;  //子节点需要继承父节点的累加值
        tree[p*2+1].flag += tree[p].flag;
        tree[p].flag = 0;   //操作结束后将父结点的标记 清0
    }
}
```

结合单点修改，单点查询，懒惰标记，我们通过代码来看下 **标记** 下发的实现。

```
void change(int p,int l,int r,int k) {
    if(l<=t[p].l&&r>=t[p].r) {  //完全重叠
        t[p].s += k*(t[p].r - t[p].l+1);    //整个区间都+k
        t[p].add+=k;            // 标记 k
        return;
    }
    add_flag(p);        //标记代码，
    int mid = (t[p].l+t[p].r)/2;
    if(l<=mid)  change(p*2,l,r,k);  //左节点存在重叠
    if(r>mid)   change(p*2+1,l,r,k);    //右节点存在重叠
    t[p].s = t[p*2].s+t[p*2+1].s;
}
LL ask(int p,int l,int r) {
    if(l<=t[p].l && r>=t[p].r)  return ans+=t[p].s;     // 情况2：不需要下发标记，直接return
    add_flag(p);    //递归查找值时，查询是否有标记，有的话则只更新走过路径的数值

    int mid=(t[p].l+t[p].r)/2;
    if(l<=mid)  ask(p*2,  l,r);
    if(r> mid)  ask(p*2+1,l,r);
    return ans;
}
```

P3372 线段树1

算法分析：

本题是**区间修改 + 区间查询。**

代码参考：

```
#include<iostream>
#include<cstdio>
using namespace std;
const int M=100010;
long long a[M];
struct node {
    int left,right; //左右节点
    long long value,flag;   //节点数值和标记
}tree[4*M];         //线段树组要开4倍大小

inline long long read() {   //读入优化
    char ch;
    while((ch=getchar())<'0'|| ch>'9');
    int res = ch-'0';
    while((ch=getchar())>='0'&&ch<='9') res = res*10+ch-'0';
    return res;
}

//建树
void build(int p,int L,int R) {
    tree[p].left = L,   tree[p].right = R;
    //递归出口
    if(L==R) {
        tree[p].value = a[L];
        return ;
    }
    else {
        int mid = (L+R)/2;
        build(p*2,L,mid);
        build(p*2+1,mid+1,R);
        tree[p].value = tree[p*2].value + tree[p*2+1].value;
    }
}
//区间修改--懒惰标记值，
void spread(int p) {
    if(tree[p].flag) {  //如果该节点有标记，则进行该区间的更新
        tree[p*2].value += tree[p].flag *(tree[p*2].right - tree[p*2].left
+1);
        tree[p*2+1].value += tree[p].flag *(tree[p*2+1].right -
tree[p*2+1].left +1);
        tree[p*2].flag += tree[p].flag;  //子节点需要继承父节点的累加值
        tree[p*2+1].flag += tree[p].flag;
        tree[p].flag = 0;   //操作结束后标记为0
    }
}
//区间修改
void change(int p,int x,int y,int z) {
    if(x<=tree[p].left && y>=tree[p].right) {   // 区间修改大于x-y，则直接把
[x,y]*z
        tree[p].value += (long long)z*(tree[p].right-tree[p].left+1);
```

```
        tree[p].flag+=z;     //打上标记
        return ;
    }
    spread(p);  //下发给子节点
    int mid = (tree[p].left + tree[p].right)/2;
    if(x<=mid)  change(p*2,x,y,z);  //查询区间在左半区
    if(y>mid)   change(p*2+1,x,y,z);     //查询区间在右半区
    tree[p].value = tree[p*2].value + tree[p*2+1].value;
}
//区间查询
long long query(int p,int x,int y) {
    if(x<=tree[p].left&&y>=tree[p].right)   return tree[p].value;
    spread(p);  //注意懒惰标记的下发
    int mid=(tree[p].left + tree[p].right)/2;
    long long ans=0;
    if(x<=mid) ans+=query(p*2,x,y);
    if(y>mid)  ans+=query(p*2+1,x,y);
    return ans;
}
int main() {
    int n,m;
    n=read();   m=read();
    for(int i=1;i<=n;i++){
        a[i] = read();
    }
    build(1,1,n);
    for(int i=1;i<=m;i++) {
        int q,x,y,z;
        q = read();
        if(q == 1) {
            x=read(),   y=read(),   z=read();
            change(1,x,y,z);
        }
        else {
            x=read(),   y=read();
            cout<<query(1,x,y)<<endl;
        }
    }
    return 0;
}
```

作为初学者在写线段树的时候，写一个功能函数后就测试一个功能函数，减少一些不必要的干扰。

P3373 线段树 2

代码参考：

```
#include<cstdio>
using namespace std;
typedef long long ll;
const int N=100010;
ll a[N],ans;
int n,m,mod;
struct node {
    int l,r;
    ll s,add,mul;
}t[4*N];
```

```
void build(int p,int l,int r) {
    t[p].add = 0;    t[p].mul = 1;  //初始化标记
    t[p].l = l, t[p].r = r;
    if(l==r){
        t[p].s = a[l];
        return ;
    }
    int mid = (l+r)/2;
    build(p*2,l,mid);
    build(p*2+1,mid+1,r);
    t[p].s = (t[p*2].s + t[p*2+1].s)%mod;
}
void flag(int p) {
    //先进行乘法标记
    t[p*2].mul = (t[p*2].mul*t[p].mul)%mod;
    t[p*2+1].mul = (t[p*2+1].mul*t[p].mul)%mod;
    t[p*2].add = (t[p*2].add*t[p].mul)%mod;
    t[p*2+1].add = (t[p*2+1].add*t[p].mul)%mod;
    t[p*2].s = (t[p*2].s*t[p].mul)%mod;
    t[p*2+1].s = (t[p*2+1].s*t[p].mul)%mod;
    t[p].mul =1;
    //进行加法标记
    t[p*2].add = (t[p*2].add+t[p].add)%mod;
    t[p*2+1].add = (t[p*2+1].add+t[p].add)%mod;
    t[p*2].s = (t[p*2].s + t[p].add*(t[p*2].r-t[p*2].l+1))%mod;
    t[p*2+1].s = (t[p*2+1].s + t[p].add*(t[p*2+1].r-t[p*2+1].l+1))%mod;
    t[p].add=0;

}
void add_change(int p,int l,int r,int x) {
    if(l<=t[p].l && r>=t[p].r) {    //完全覆盖
        t[p].add=(t[p].add+x)%mod;
        t[p].s=(t[p].s+(t[p].r-t[p].l+1)*x)%mod;
        return ;
    }
    flag(p);    //下传标记
    int mid=(t[p].l+t[p].r)/2;
    if(l<=mid)  add_change(p*2,l,r,x);
    if(r>mid)   add_change(p*2+1,l,r,x);
    t[p].s = (t[p*2].s+t[p*2+1].s)%mod;
}
void mul_change(int p,int l,int r,int x) {
    if(l<=t[p].l && r>=t[p].r) {
        t[p].mul = (t[p].mul*x)%mod;
        t[p].add = (t[p].add*x)%mod;
        t[p].s = (t[p].s*x)%mod;
        return ;
    }
    flag(p);    //下传标记
    int mid=(t[p].l+t[p].r)/2;
    if(l<=mid)  mul_change(p*2,l,r,x);
    if(r>mid)   mul_change(p*2+1,l,r,x);
    t[p].s = (t[p*2].s+t[p*2+1].s)%mod;
}
ll ask(int p,int l,int r) {
    if(l<=t[p].l && r>=t[p].r)  return ans = (ans+t[p].s)%mod;
    flag(p);    //下传标记
    int mid=(t[p].l+t[p].r)/2;
```

```
    if(l<=mid)  ask(p*2,  l,r);
    if(r> mid)  ask(p*2+1,l,r);
    return ans%mod;
}
int main() {
    scanf("%d %d %d",&n,&m,&mod);
    for(int i=1;i<=n;i++)   scanf("%lld",&a[i]);
    build(1,1,n);   //建树
    for(int i=1;i<=m;i++) {
        int q,x,y,z;
        scanf("%d",&q);
        ans=0;      //注意：每次查询就将ans重置为0 ，否则下一次查询结果会出错
        if(q==1){
            scanf("%d %d %d",&x,&y,&z);
            mul_change(1,x,y,z);
        }
        if(q==2) {
            scanf("%d %d %d",&x,&y,&z);
            add_change(1,x,y,z);

        }
        if(q==3) {
            scanf("%d %d",&x,&y);
            ans=0;
            printf("%lld \n",ask(1,x,y));
        }
    }
    return 0;
}
```

树状数组

前置知识：

lowbit 运算

如何快速求出一个二进制数位数最低的1在哪?

$lowbit$（n）定义为非负整数n在二进制表示下“最低位的1及其后面所有的0”构成的数值。例如：$n=10$ 的二进制表示为 $(1010)_2$，则$lowbit(n)=2=(10)_2$。

设$n>0$，n的第k位是 1 ，第$0\sim(k-1)$ 位都是0。为了实现$lowbit$ 运算，先把 n 取反，此时第k 位变为0，第$0\sim(k-1)$位都是1。再令$n=n+1$，此时因为进位，第k位变为1，第$0\sim(k-1)$位都是0。

在上面的取反加1操作后，n的第$k+1$到最高位恰好与原来相反，所以$n\&(\sim n+1)$仅有第k位为1，其余位都是0。而在补码表示下，$\sim n=-1-n$，因此：

$$lowbit(n)=n\&(\sim n+1)=n\&(-n)$$

$lowbit$ 运算**可以找出整数二进制表示下所有是1的位**，所花费的时间与 1 的个数同级。算法过程：例如：$n=9=(1001)_2$

- $9-lowbit(9)=9-1=8=(1000)_2$，此时$n=8$
- $8-lowbit(8)=8-8=0$，此时 $n=0$
- 当$n=0$是退出循环。

代码参考：

```
int lowbit(n){    return n&(-n);}//计算整数二进制下所有是1的位while(n){    n = n-lowbit(n);}
```

树状数组

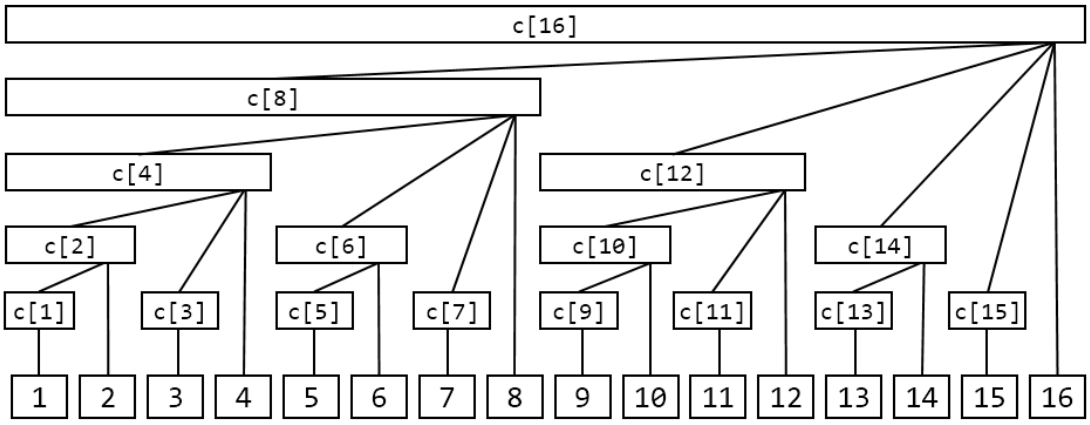
树状数组（Binary Indexed Tree，BIT）是一种利用数的二进制特征进行检索的树状结构。树状数组是一种奇妙的数据结构，不仅非常高效，而且代码极其简洁。其基本用途是维护序列的前缀和。

首先，我们将一个大区间拆分成$logn$ 个小区间。比如：$[1,7]$ 拆分成： $[1,4]$, $[5,6]$,$[7,7]$。长度对应的是$lowbit(4)=4, lowbit(6)=2, lowbit(7)=1$。对于给定的序列 a ，我们建立一个数组 c，其中$c[x]$ 保存序列 a 的区间$[x-lowbit(x)+1,x]$ 中所有数的和。

事实上，数组 c 可以看作一个如下图的树形结构。 该结构满足以下性质：

1. 每个内部结点$c[x]$ 保存以它为根的子树中所有叶结点的和
2. 每个内部结点$c[x]$的子节点个数等于$lowbit(x)$的个数
3. 除树根外，每个内部结点$c[x]$ 的父结点是$c[x+lowbit(x)]$
4. 树的深度为 $O(logN)$。

如果N不是 2 的整次幂，那么树状数组就是一个具有同样性质的森林结构。



区间数组 c[ ] 的求解图

$c[1]=a[1]$，　$c[2]=a[1]+a[2]$，　$c[3]=a[3]$，　$c[4]=a[1]+a[2]+a[3]+a[4]$ 。这个过程的参考代码如下：

```
void add(int x,int y) { //序列a[x] = y
    c[x] = y;
    for(int i=x-lowbit(x)+1;i<x;i++)
        c[x] += a[i];
}
```

树状数组支持的基本操作有两个：

1. 查询前缀和，即序列 a 第 1 ~ x 个数的和。求解过程为：先求出x 的二进制表示中每个等于 1 的位，把$[1,x]$分成$O(logN)$ 个小区间，而每个小区间的区间和都已经保存在数组c 中。

如果要求解$[1,7]$。其实就是$c[7]+c[6]+c[4]$。

参考代码

```
int ask(int x) {
    int ans=0;
    while(x) {
        ans+=c[x];
        x= x- lowbit(x);
    }
    return ans;
}
```

当然，若要查询序列a 的区间$[l,r]$中所有数的和，只需计算$ask(r)-ask(l-1)$即可。

2. 单点修改，给序列中的一个数$a[x]$增加y，同时正确维护序列的前缀和。只有结点$c[x]$及其所有祖先结点保存的"区间和"包含$a[x]$，而任意一个结点的祖先至多只有$logN$个，我们逐一对它们的c 值进行更新即可。

```
void change(int x,int y){
    while(x<=n) {   //往上修改的结点不超过n
        c[x]+=y;
        x=x+lowbit(x);  //往父结点走
    }
}
```

P3374 树状数组1

代码参考

```
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N=500010;
int a[N],c[N];
int n,p;
int lowbit(int x) { return x& -x;}  //树状数组的计算大部分都是基于lowbit()

void add(int x,int y) { //创建c数组
    c[x]=y;
    for(int i=x-lowbit(x)+1;i<x;i++)    c[x]+=a[i];
}

void change(int x,int y) {  //单点修改
    while(x<=n) {
        c[x]+=y;
        x= x+lowbit(x);
    }
}
int ask(int x) {        //前缀和
    LL ans=0;
    while(x) {
        ans += c[x];
        x-= lowbit(x);
    }
    return ans;
}
```

```
28
29  int main() {
30      scanf("%d%d",&n,&p);
31      for(int i=1;i<=n;i++) {
32          scanf("%d",&a[i]);
33          add(i,a[i]);          //创建c数组
34      }
35
36      while(p--) {
37          int op,x,y; scanf("%d%d%d",&op,&x,&y);
38          if(op==1) {
39              change(x,y);
40          }
41          else if(op==2) {
42              cout<<ask(y)-ask(x-1)<<endl;
43          }
44      }
45      return 0;
46  }
```

扩展：离散化

建二叉树是线段树的基本操作，但是二叉树的大小并不是无限制，例如规模 $100,000,000$以上的二叉树会超过允许的存储空间。在竞赛中我们不能直接建这么大的二叉树，此时可以用 “**离散化**”这种小技巧来解决。

例如：我们有区间$[1,3],[2,5],[3,8],[2,100]$，假设每个区间表示一种颜色，问最终能看到多少种颜色?

离散化的过程如下：

1. 提取这四个区间的端点：$1,3,2,5,3,8,2,100$
2. 排序并且删除相同的端点，得到：$1,2,3,5,8,100$
3. 把原线段的端点映射到新的线段上：

原端点	1	2	3	5	8	100
映射后	1	2	3	4	5	6

新的4个区间为$[1,3],[2,4],[3,5],[2,6]$。我们可以发现，离散化后对结果不会影响。

离散化的压缩比是很可观的。例如原线段树的区间长度为：$10,000,000$，而其中真正用到的子区间是$100,000$，那么子区间的端点最多有$200,000$个。经过离散化压缩后，新的线段树区间是$200,000$，压缩率是$200000/100000000=2\%$

伪代码

```
1
2  bool cmp(node a,node b){
3      return a.val<b.val;
4  }
5
6  struct node{
7      int val;
8      int id;
```

```
}a[N];       //a为原端点

int b[N];   //b为映射端点

for(int i=1;i<=n;i++){
    cin>>a[i].val;  a[i].id=i;
}
sort(a+1,a+1+n,cmp);
for(int i=1;i<=n;i++) {
    b[a[i].id] = i;
}

```

离散化题目：P1955 [NOI2015] 程序自动分析

相关资料：离散化入门

应用

P1908 逆序对 / P1774 最接近神的人

题目描述

逆序对的定义：对于给定的一段正整数序列，逆序对就是序列中$a_i > a_j$且$i < j$的有序对。给定一段正整数序列，求逆序对的数目。注意序列中可能有重复数字。

输入输出格式

第一行，一个数n，表示序列中有n个数。

第二行n个数，表示给定的序列。序列中每个数字不超过10^9。

输出序列中逆序对的数目。

输入#1	输出#1
6 5 4 2 6 3 1	11

朴素算法——$25pts$

朴素算法1： 直接做两个 `for` 循环，如果存在逆序对就累加即可。

朴素算法2：

采用桶排序的思路。**将元素的值当做桶的编号**。我们知道桶处理数据时，是要把所有桶都跑一遍的。我们发现，本题的数值10^9，绝对会**超空间**。我们还发现，数值的个数如果在10^6内，这时，我们可以用离散化的方式，把**大数离散成小数**，把桶控制在10^6。

桶的问题解决后，接着下一步。怎么统计逆序对呢？ 其实很简单。从小到大跑一遍桶，当前桶为i，当前遍历的数为x，如果$i > x$则存在逆序对。累加即可。

两者总的时间复杂度为$O(N^2)$

```
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N=500010;

```

```cpp
struct node{
    int val,id;
}b[N];
int a[N];
bool vis[N];    //标记桶
int n;

bool cmp(node a,node b) {
    if(a.val == b.val)  return a.id>b.id;
    return a.val < b.val;
}

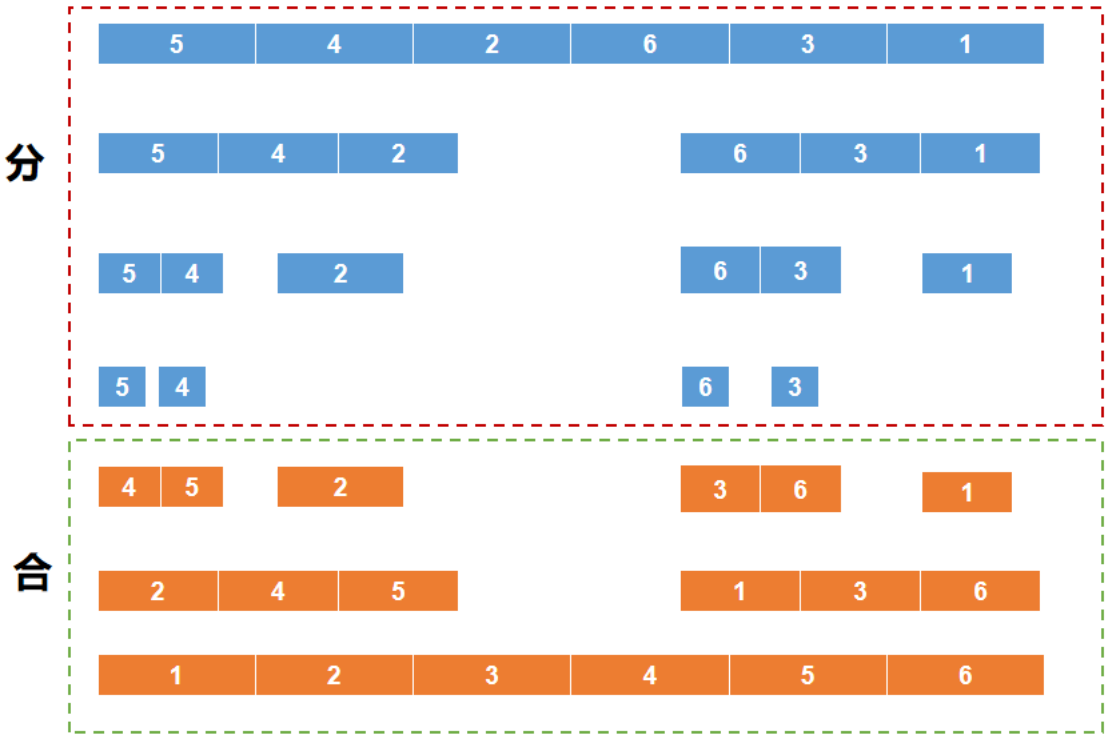
int main() {
    cin>>n;
    for(int i=1;i<=n;i++) {
        cin>>b[i].val;  b[i].id=i;
    }
    sort(b+1,b+1+n,cmp);
    //离散化
    for(int i=1;i<=n;i++)   a[b[i].id] = i;
    LL ans=0;
    for(int i=1;i<=n;i++) {
        int x=a[i]; //当前出现的数
        vis[x]=1;   //标记该数出现了
        for(int j=x+1;j<=n;j++) {   //如果x后面的数出现过就累加ans
            if(vis[j])  ans++;  //假设在5之前就出现了6，那就存在一对逆序对
        }
    }
    cout<<ans;
    return 0;
}

```

算法1： 归并排序

朴素算法1 的时间复杂度是$O(N^2)$，我们看能不能用$O(nlog\ n)$ 的。这种复杂度的处理，可能是分治算法，或者树形数据结构。

我们看用分治算法分析一下：$5,4,2,6,3,1$。



我们直接来分析下 $[2,4,5],[1,3,6]$ 是怎么归并计算逆序对的。我们来模拟一下归并的过程。

如果$a[1]=2>a[4]=1$，则意味着$2,4,5$ 能和 1 组成逆序对， 对数为 3； 接着右边移动一位

如果$a[1]=2<a[5]=3$， 则意味着 2 不能和右边的组成逆序对了，对数为 0； 接着左边移动一位

如果$a[2]=4>a[5]=3$， 则意味$4,5$ 能和 3 组成逆序对，对数为 2； 接着右边移动一位

如果$a[2]=4<a[6]=6$， 则意味着 3 不能和右边的组成逆序对了，对数为0,；接着左边移动一位。

如果$a[3]=5<a[6]=6$，对数为 0。

如果区间$[2,4,5],[1,3,6]$ 合并过程会有 5对。

同理我们知道 合并$[2,4,5]$ 时，逆序对有 2对。 合并$[1,3,6]$ 时，逆序对有 2 对。

合并$[4,5]$ 时，逆序对有 1对； 合并$[3,6]$时，逆序对有 1 对。 总共有：
$ans=1+1+2+2+5=11$。

那么规律是发生在归并过程的$a[l]>a[r]$ 时，会出现逆序对。那么对数为： $mid+1-l$ （即左边有多少个数能和当前的$a[r]$ 组成逆序对）。

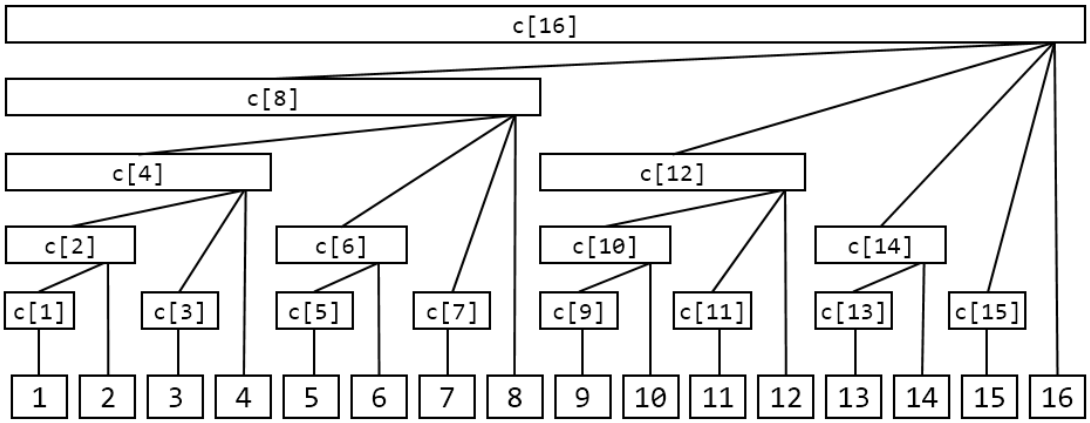
```
1
2  #include<bits/stdc++.h>
3  using namespace std;
4  const int N=100010;
5  int a[N],b[N];
6  int n;
7  long long ans=0;
8
9  void msort(int le, int ri) {
10     int mid = (le+ri)/2;
11     if(ri>le) {     //分治过程
12         msort(le,mid);
13         msort(mid+1,ri);
14     }
```

```
15
16      int i=le, j=mid+1,k=0;
17      while(i<=mid && j<=ri) {
18          if(a[i]>a[j]) {      //如果出现逆序对，则统计 mid+1-i 即左边有多少个数能和第
    a[j]个组成逆序
19              b[++k] = a[j++];
20              ans += mid+1-i;
21          }
22          else    b[++k] = a[i++];
23      }
24
25      while(i<=mid) {
26          b[++k] = a[i++];
27      }
28      while(j<=ri)    b[++k] = a[j++];
29      for(int i=le,k=1; i<=ri; i++,k++) {
30          a[i] = b[k];
31      }
32  }
33  int main() {
34      cin>>n;
35      for(int i=1;i<=n;i++)   cin>>a[i];
36      msort(1,n);
37      //for(int i=1;i<=n;i++) cout<<a[i]<<" ";
38      cout<<ans;
39      return 0;
40  }
```

算法2：树状数组

从算法 1 的朴素算法中，我们知道，我们每遍历到$a[i]$ 时，我们就需要再$[a[i]+1,n]$ 这个范围内找比$a[i]$出现的早有多少个数（即可得到逆序对）。但这块的时间复杂度近乎$O(N)$。这里我们使用树状数组，尝试把这块优化到$O(logN)$。这样总的时间复杂度就能接受了。

- 第一个问题：离散化的处理，如果数据是：$1,2,2,1,3,4$。 如果数字相同，出现越早，映射的数字越小。 即映射成：1，3，4，2，5，6。 观察可知： 离散后的数字对最终结果的逆序对是没有影响的。
- 第二个问题：统计第 i 个数 与$[1,b[i]]$ 个数构成了多少逆序对。这也是我们要优化的（通过树状数组优化）。
- 我们初始化树状数组$c[N]=0$。$C[i]$ 表示在$c[i]$ 范围内比i 小的数有多少个。那么$[1,b[i]]$的值可以通过树状数组的$ask()$计算前缀和来实现。 最后组成的逆序对为：$i-ask(b[i])$（总数 - 小于$b[i]$的个数 = 大于$b[i]$ 的个数）



区间数组 c[ ] 的求解图

```cpp
#include<bits/stdc++.h>
#define ull unsigned long long
#define N 500010
using namespace std;
int b[N];
struct node{
    int num,val;
}a[N];
int c[N];   //树状数组
int n;
ull ans=0;

bool cmp(node a,node b) {   //如果数字相同的话，出现的越早映射后的数字就小即可
    if(a.val == b.val)  return a.num<b.num;
    return a.val<b.val;
}

inline int lowbit(int x){return x&(-x);}

void change(int x,int y) {
    while(x<=n) {
        c[x]+=y;
        x+= lowbit(x);
    }
}

ull ask(int x) {
    ull s= 0;
    while(x) {
        s += c[x];
        x-=lowbit(x);
    }
    return s;
}
int main() {

    cin>>n;
    for(int i=1;i<=n;i++) {
        a[i].num = i;
        cin>>a[i].val;
    }
```

```
    sort(a+1,a+1+n,cmp);     //时间复杂度在O(NlogN)
    for(int i=1;i<=n;i++) {
        b[a[i].num] = i;     //把a数组的值从小到大映射成 1，2,3,4，...,n
    }

    for(int i=1;i<=n;i++) {
        change(b[i],1);
        ans+= i - ask(b[i]);
    }
    cout<<ans;
    return 0;
}

```

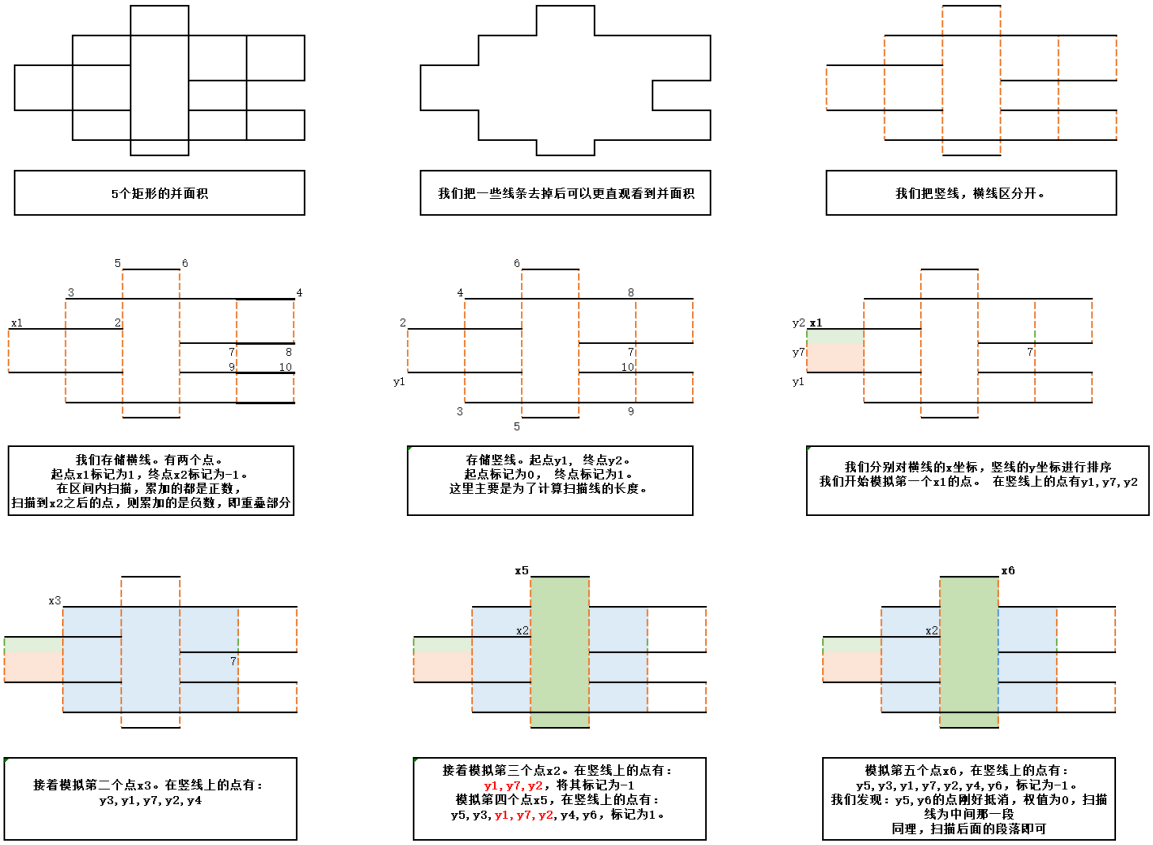
算法3：线段树

线段树的思路与树状数组类似。同学们自行编写代码

P5490 【模板】扫描线

题目描述：求 n个矩形的面积并。

题目分析：



算法实现：

我们将**线转化为点**的问题，N 条竖线，最多转化为$2N-1$ 个点， 同理，N 点横线，最多转化为$2N-1$ 个点。

我们建一个结构体，记录当前点的性质：x,to,e，分别为坐标值， 边的编号， 入边/出边

另a 数组为横线， b数组为竖线

由于数据太大，我们将坐标值进行离散化处理。这里我们主要离散化竖线的数组，记为c数组

算法过程：

- 我们分别对横线和竖线进行排序。接着对竖线b进行离散化
- 接着从小到大遍历a数组， 另当前的横坐标为$a[i]$， 我们获取其编号j， 接着获取当前边对应的$y1, y2$排序后所在的编号x, y。我们将$[x, y]$范围内的y坐标都打上标记，如果扫描的横坐标为入边，则标记1， 否则标记-1；
- 接着扫描当前纵坐标y，如果是非0， 则累加其长度len， 如果是0，则表示该坐标的面积被抵消，不用再累加了。
- 最后将当前累加的len * 当前横线的长度 （$a[i+1].x - a[i].x$）。

从图上分析可知，总的时间复杂度是$O(n^2)$

代码参考： $20pts$

```
// 本代码参考《算法进阶指南》
#include<bits/stdc++.h>
#define ll long long
using namespace std;
const int N=200010;

struct node{
    int x;
    int to;
    int e;
}a[N],b[N]; //a表示横线， b表示竖的扫描线
int c[N][2];    //存储b线，0表示入边，1表示出边
int f[N];   //记录该面积扫描的次数
ll ans;
/*int i,j,k,x,y,z,len;
int x1,y1,x2,y2;*/
bool cmp(node a,node b){return a.x<b.x;}

int main() {
    int n;
    scanf("%d",&n);
    for(int i=1;i<=n;i++) {
        int x1,y1,x2,y2;    scanf("%d%d%d%d",&x1,&y1,&x2,&y2);
        a[i*2-1].x = x1;
        a[i*2-1].to = i;
        a[i*2-1].e = 1;
        a[i*2].x = x2;  a[i*2].to = i;  a[i*2].e=-1;    //出边后面要减掉的

        b[i*2-1].x = y1;
        b[i*2-1].to = i;
        b[i*2-1].e = 0;
        b[i*2].x = y2;  b[i*2].to = i;  b[i*2].e=1;
    }
    sort(a+1,a+1+2*n,cmp);  //横线按x坐标排序
    sort(b+1,b+1+2*n,cmp);  //竖线按y坐标排序

    for(int i=1;i<=n*2;i++) c[b[i].to][b[i].e] = i; //离散

    for(int i=1;i<2*n;i++) {    //遍历每一条横线
        int j=a[i].to;
        int x=c[j][0];  //y起点的位置
        int y=c[j][1];  //y终点的位置
```

```
        int z=a[i].e;   //该横线的性质，入边为1， 出边为-1
        for(int k=x;k<y;k++)    f[k]+=z;    //正数累加，负数则累减（重复的减去），
        ll len=0;
        for(int k=1;k<2*n;k++) {    //遍历竖线，即扫描线
            //只要非0即计算len
            if(f[k])    len+=b[k+1].x - b[k].x;
        }
        ans+=len*(a[i+1].x-a[i].x);
    }
    cout<<ans;
    return 0;
}

```

线段树优化算法

算法优化：从以上扫描图可知，我们每次都需要花费$O(n)$的复杂度在计算len上，我们可以观察，其数据类型与线段树相似。我们可以想办法通过线段树来维护len的长度，将复杂度优化到$O(logn)$，这样总的复杂度就是$O(nlogn)$了。

```
// 本代码参考洛谷第一篇题解
#include<iostream>
#include<cstdio>
#include<algorithm>
#define ll long long
using namespace std;
const int N=200010; //线转点，需要开2倍空间

int n,x1,y1,x2,y2;
int Lx[N];  //横线
struct Line{
    int l,r,h;
    int flag;
    bool operator<(const Line &a) const{return h<a.h;}
}Ly[N];     //竖线
struct node{
    int l,r,s;
    ll len;
}t[N*4];    //线段树需要开4倍大小

//建树
void build(int p,int l,int r){
    t[p].l = l, t[p].r=r;   t[p].s=0, t[p].len=0;
    if(l==r)    return ;
    int mid=(l+r)/2;
    build(p*2,l,mid);
    build(p*2+1,mid+1,r);
    return ;
}
//类似下传标记，更新长度
void push_up(int p) {
    int l=t[p].l, r=t[p].r;
    if(t[p].s){
        t[p].len = Lx[r+1]-Lx[l];   //更新宽度
    }
    else{
        //如果已经遍历到叶子节点，则直接返回0，否则将累加两个子节点的len
```

```
        t[p].len = t[p].l==t[p].r?0:t[p*2].len+t[p*2+1].len;
        //t[p].len = t[p*2].len+t[p*2+1].len;
    }
}

void change(int p,int l,ll r,int f) {
    int x=t[p].l, y=t[p].r;
    if(Lx[y+1]<=l || r<=Lx[x])  return ;    //如果当前的横线不在该范围内，则直接退出即可
    if(l<=Lx[x] && r>=Lx[y+1]) {    //扫描的当前节点在[l,r]范围内
        t[p].s+=f;
        push_up(p); //更新该点覆盖的宽度
        return ;
    }
    change(p*2,  l,r,f);
    change(p*2+1,l,r,f);
    push_up(p);
    return ;
}
int main() {
    scanf("%d",&n);
    for(int i=1;i<=n;i++) {
        scanf("%d%d%d%d",&x1,&y1,&x2,&y2);
        Lx[i*2-1] = x1;
        Lx[i*2]   = x2;
        Ly[i*2-1] = (Line){x1,x2,y1,1};
        Ly[i*2]   = (Line){x1,x2,y2,-1};
    }
    n=n*2;
    sort(Lx+1,Lx+1+n);
    sort(Ly+1,Ly+1+n);
    int m = unique(Lx+1,Lx+1+n)-Lx-1;   //去重
    build(1,1,m-1);
    ll ans=0;
    for(int i=1;i<n;i++) {
        change(1,Ly[i].l, Ly[i].r, Ly[i].flag); //遍历当前竖线
        ans+= t[1].len* (Ly[i+1].h - Ly[i].h);
    }
    cout<<ans;
    return 0;
}
```

扩展知识： 线段树动态开点。（参考：《算法进阶指南——线段树》）

总结：

	树状数组	线段树
优点	效率高、代码短	效率较高、扩展性好
缺点	不易扩展、不太直观	代码较长、直观性一般

例题

P2068 统计和

P1531 I Hate It

P2880 [USACO07JAN]Balanced Lineup G

P5057 [CQOI2006]简单题

P3870 [TJOI2009]开关

P1816 忠诚

P4513 小白逛公园

P3368 【模板】树状数组 2

P6225 [eJOI2019]异或橙子

P1966 火柴排队

P3605 [USACO17JAN]Promotion Counting

P4054 [JSOI2009]计数问题

P4588 [TJOI2018]数学计算

P1856 [USACO5.5]矩形周长Picture

扩展

关于树形数据结构，再深入的知识点有：二维线段树，二维树状数组，平衡二叉树，基环树。这些都是省选常考的内容。感兴趣的同学可以提前了解下。